

BHTTP/1: HTTP, in binary

BHTTP carries HTTP requests and responses over one long-lived TCP connection, using length-prefixed binary frames instead of text lines. The client asks, the server answers in order, and the connection stays open between requests. These two pages are all you need to write either side. Methods, status codes and header values mean exactly what they mean in HTTP (RFC 9110). This document only defines how they are carried. MUST, SHOULD and MAY are as in RFC 2119. Integers are unsigned and big-endian; Length (16) is a 2-byte field.

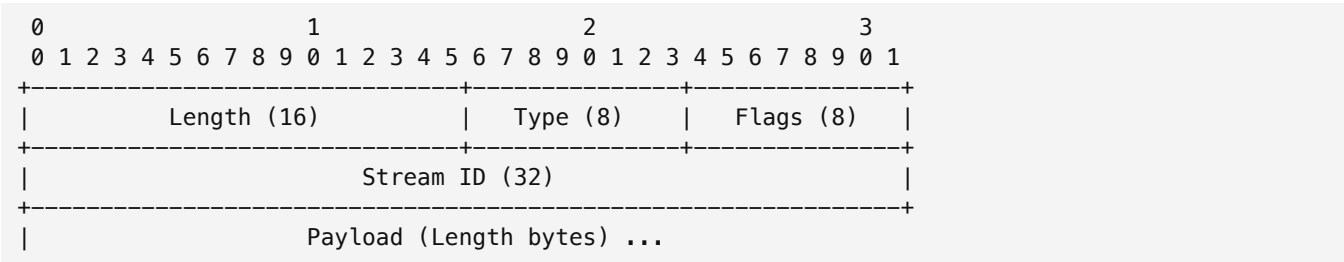
1. Connection

The client connects and sends an 8-byte preface, 42 48 54 50 2F 31 0D 0A (BHTTP/1\r\n). After that, both directions carry nothing but frames. The server sends no preface.

A connection carries any number of requests, and neither side closes it because a response is finished. A client MAY send a request before the previous response is complete. The server answers strictly in the order requests arrive, and the frames of two responses are never interleaved. Either side MAY close the connection between frames. A side that closes because of an error or a timeout SHOULD send GOAWAY first.

2. Frames

Every frame is an 8-byte header followed by Length bytes of payload:



- Length:** payload size, 0–65535. The 8 header bytes are not counted.
- Type:** what the payload is (table below).
- Flags:** 0x01 END_STREAM means this is the last frame of its message. Every other bit MUST be sent as 0 and MUST be ignored on receipt.
- Stream ID:** the request this frame belongs to; 0 means the connection itself.

Type	Name	Sent by	Stream ID	Payload
0x01	REQUEST	client	new, above 0	method, path, header fields (§3)
0x02	RESPONSE	server	its request's	status, header fields (§3)
0x03	DATA	both	its message's	body bytes
0x04	GOAWAY	both	0	error code (16), then an optional UTF-8 reason

A receiver that meets a frame type it does not know MUST read and discard exactly Length payload bytes and carry on as if the frame had never been sent. It does not reply, close the connection, or count the frame as part of any message, whatever its flags or stream. This is how later versions add features without breaking v1 peers.

A message is one REQUEST or RESPONSE frame followed by zero or more DATA frames on the same stream. Its last frame carries END_STREAM, so a REQUEST or RESPONSE with END_STREAM has no body. Each request uses a stream ID larger than any the client has used on this connection, and the response carries its request's ID.

3. Header blocks

REQUEST payload: Method (8) | Path length (16) | Path | header fields...
RESPONSE payload: Status (16) | header fields...

Method is one of 1 GET, 2 HEAD, 3 POST, 4 PUT, 5 DELETE; other values are unassigned. **Path** is the request target as it would appear in an HTTP/1.1 request line. It starts with /, may be percent-encoded and may end in ?query. **Status** is the final HTTP status, 200–599. There are no 1xx responses in v1, and no reason phrase.

Header fields fill the rest of the payload, back to back, and the block ends where the payload ends. The first byte of each field says how its name is given:

1iiiiiii	Value length (16)	Value	indexed name, i = 1..127
0nnnnnnn	Name (n bytes)	Value length (16)	Value literal name, n = 1..127

1	2	3	4	5	6	7	8	9	10
host	user-agent	accept	if-none-match	server	date	content-type	content-length	last-modified	etag

- Senders **SHOULD** use the index for these ten names, **MUST** spell out every other name, and **MUST NOT** send an index above 10. Receivers **MUST** accept a table name in either form.
- **A receiver that meets an index from 11 to 127 MUST skip that field and keep going.** Its value is length-prefixed, so this is always possible: the same rule as for frames.
- Names are lowercase tokens: a–z 0–9 ! # \$ % & ' * + - . ^ _ ` | ~. Values **MUST NOT** contain CR, LF or NUL.
- A header block **MUST** fit in one frame. There is no continuation frame.
- `content-length`, when sent, **MUST** equal the total DATA bytes of the message. The exceptions are a response to HEAD and a 304, where it gives the size the body would have had.

4. Serving files

The server drops any `?query`, percent-decodes the rest, and only then checks it and maps it to a file under its root directory. `/` and any directory mean that directory's `index.html`. A hit is answered **200** with `server`, `date`, `content-type`, `content-length`, `last-modified` and `etag`, then the file in DATA frames. Every other outcome gets the status below and a short text/plain body saying why (no body for HEAD or 304).

Status	When
304	<code>if-none-match</code> lists the file's current etag, or is *
400	the request is malformed (§5), or the path has a <code>..</code> segment, a bad % escape, or <code>%00</code>
403 / 404	unreadable / missing. Anything resolving outside the root (via symlinks) is 404
405	POST, PUT or DELETE. The response carries <code>allow: GET, HEAD</code> as a literal name
501	an unassigned method code

A server **MAY** answer before a request body ends, and **MUST** discard any DATA it does not need.

5. Errors

Malformed request: answer 400 and keep the connection open. A REQUEST is malformed when its payload does not parse: shorter than 3 bytes, a length that runs past the end of the payload, an empty path or one not starting with `/`, a field byte of `0x80` (index 0) or `0x00` (empty name), a name that is not a lowercase token, or CR/LF/NUL in a value. The frame header was fine, so the next frame is still where Length says it is. The server answers **400** on that stream and keeps reading. Since every 16-bit Length is legal this always holds, which is why almost every error in BHTTP is recoverable.

Connection error: send GOAWAY, then close. These are cases where the sequence of frames is wrong:

- the first 8 bytes are not exactly the preface → `BAD_PREFACE`
- a REQUEST whose stream ID is 0 or not above every earlier one (even malformed) → `PROTOCOL_ERROR`
- a server receives RESPONSE, or a client receives REQUEST → `PROTOCOL_ERROR`
- a client receives RESPONSE or DATA for anything other than its oldest unanswered request, DATA before its RESPONSE or in a response to HEAD or a 304, a malformed RESPONSE, or a `content-length` that does not match the DATA received → `PROTOCOL_ERROR`

GOAWAY codes: **0** `NO_ERROR` (such as an idle timeout), **1** `PROTOCOL_ERROR`, **2** `BAD_PREFACE`, **3** `INTERNAL_ERROR`. Treat any other code as `PROTOCOL_ERROR`. The sender of a GOAWAY closes the connection right after it, and requests that were not answered are not retried on it.

6. Why these widths

HTTP/2 uses 24/8/8/31 (length, type, flags, reserved bit + stream). BHTTP uses **16/8/8/32**, 8 bytes in all.

- **Length 16.** HTTP/2 caps frames at 16 KiB unless the receiver raises the cap with SETTINGS, so its field must hold any cap a peer *might* set, up to 16 MiB. We have no SETTINGS, so whatever the field allows, every receiver must be ready to buffer. 64 KiB is a sane worst case. Bodies are split into DATA frames anyway, so it costs one 8-byte header per 64 KiB (0.012%).
- **Type 8, Flags 8.** We use 4 types and 1 flag. The rest is room for later versions.
- **Stream 32.** v1 answers in order, so today the ID only catches a response paired with the wrong request. But it is the field multiplexing needs, and 4 bytes now beats a new header layout later. HTTP/2's reserved bit keeps IDs inside a signed 32-bit int. We don't need that.
- **The 1 in the preface versions this header and nothing else.** New frame types, flags and table entries never bump it, because v1 peers skip what they don't know. Only a change to these 8 bytes would, and that is the one change skipping can't survive.